
Harnesses 101

Right now, you are the harness. This is what to replace yourself with — and what you still have to do yourself.

You are a very slow USB cable

```
# the loop, as actually performed
1 open the chat tab
2 describe the bug in prose
3 paste 40 lines of a 900-line file
4 read answer, copy the code block
5 paste into notebook, run it
6 it fails
7 copy the traceback back into the tab
8 goto 4
```

Every arrow in that loop is a human being moving text between two windows.

The model never touched your repository. It never ran your code. It never saw the traceback unless you carried it. And the moment you closed the tab, all of it was gone.

This is not a bad way to use an LLM. It is just an extremely limited one — and it is the only one many of you have tried.

What copy–paste structurally can't reach

- 01 **See your repo.** It has 40 lines out of 900. The rest it reconstructs from the average of the internet.
- 02 **Run anything.** It cannot execute your code, so it cannot know whether the answer works.
- 03 **Close its own loop.** The traceback only reaches it if you carry it there.
- 04 **Remember.** No branch, no conventions, no `pyproject.toml`, no last week.
- 05 **Be wrong cheaply.** Each wrong guess costs you a full paste–run–read cycle.

“Claude stops when the work looks done. Without a check it can run, ‘looks done’ is the only signal available, and you become the verification loop.”

Anthropic – Claude Code best practices

“It ran” is not evidence

A hallucinated method name explodes on the first execution. You will catch that one.

What you will not catch is code that runs perfectly and returns a *plausible* number: the wrong join key, the `groupby` that silently drops the nulls, the axis limit that flatters the result, the `resample` that shifts everything by one day.

In this course the deliverable is a claim about data. A chart that is confidently wrong is worse than a crash — a crash at least tells you.

“You should never trust any piece of code until you've seen it work with your own eye — or, even better, seen it fail and then fixed it.”

Simon Willison — Hallucinations in code

```
# the crash you catch
AttributeError: 'DataFrame' has no
attribute 'groupby_time'

# the one you don't
df.merge(other, on='id')    # 12,004 rows
→ silently drops 3,118 unmatched rows
```

Agent = Model + Harness

A language model, by itself, only produces text.

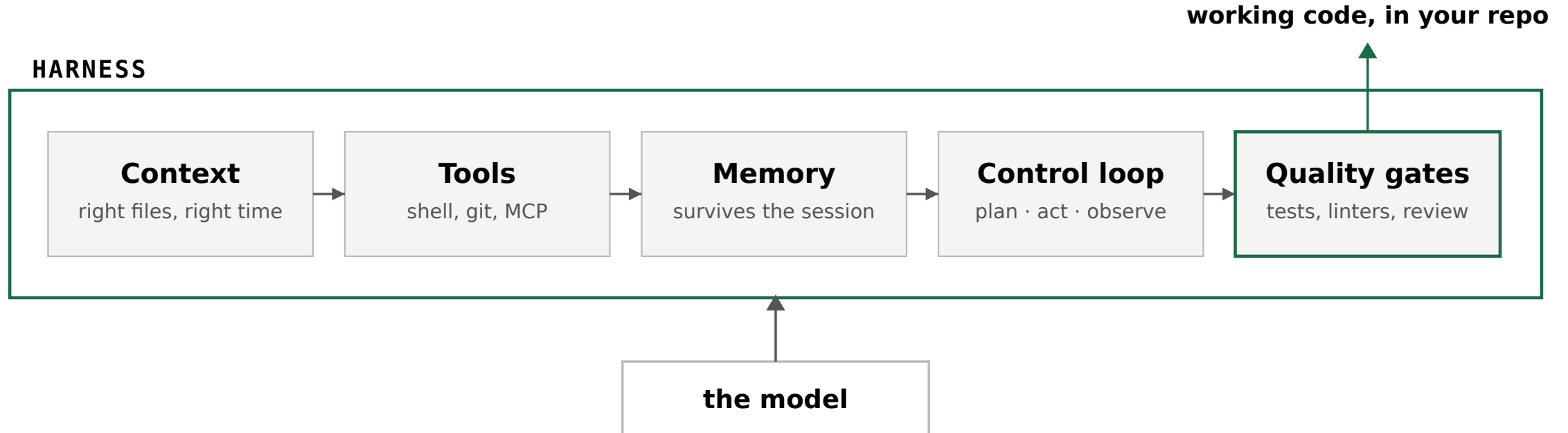
The **harness** is everything built around it that turns text into action: the files it may read, the shell it may run, the tools it may call, the memory that survives the session, and the loop that decides whether to stop or try again.

Vocabulary warning: some sources split “scaffold” (what the model sees) from “harness” (the loop that executes). Most use the two interchangeably. The concept is stable; the word is not.

“A decent model with a great harness beats a great model with a bad harness.”

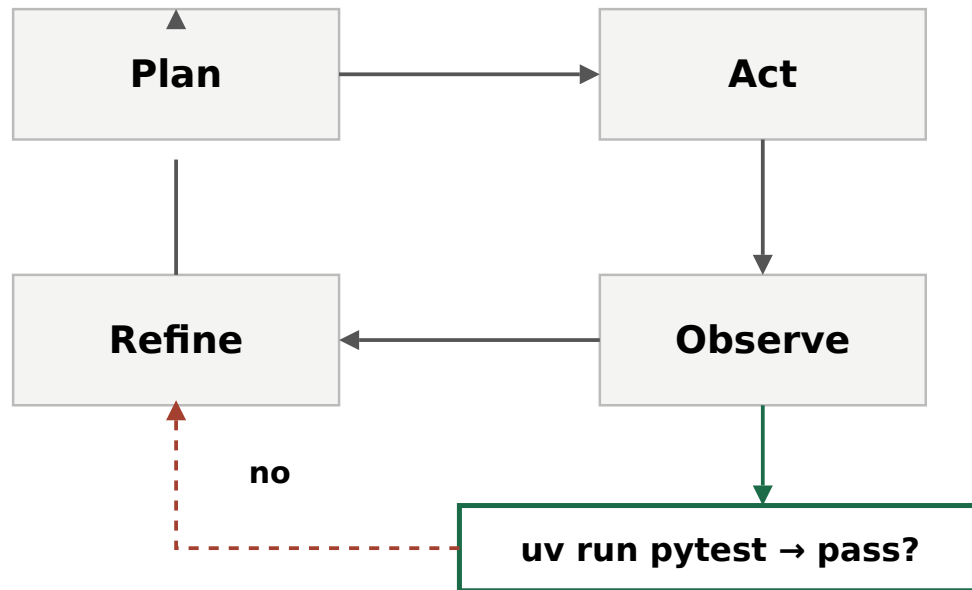
Addy Osmani – Agent harness engineering

Five parts, in every harness



Two independent explainerers — [Sam Schillace's](#) and [Pinggy's](#) — arrive at this same decomposition from different directions. That convergence is why it is worth memorising.

The loop closes without you in it



```
# no gate
model writes code
  → "looks done"
  → you run it
  → you find the bug
  → you explain the bug
```

```
# with a gate
model writes code
  → runs uv run pytest
  → reads the failure itself
  → fixes it, runs again
  → shows you passing output
```

The highest-value sentence you can add to any prompt: "run the tests and keep going until they pass."

“But who writes the check?”

Fair. If the agent writes the test from the same misreading that wrote the code, it passes for the same wrong reason. **A gate built by the thing it is gating is not a gate.**

So be honest about what the loop buys you: a harness raises the floor, not the ceiling.

Nothing that crashes survives. Nothing that is merely wrong is touched. The plausible number is still yours to catch — and always will be.

Checks that cannot be faked into passing are the ones anchored outside the code: one row you worked out by hand, a figure from the source system, last month's total.

The division of labour that works

You state the invariant. It writes the code that checks it. The invariant is the abstraction you own.

```
# things you can assert without  
# knowing the answer  
len(merged) == len(left)  
no new nulls where there were none  
total preserved across the reshape  
date range unchanged  
the key is unique where you think it is
```

Legitimate, non-circular use: ask it “*what could silently go wrong in this join?*” It generates candidates well. You decide which matter. You are the selector, not the author.

Chat window vs. harness

CHAT WINDOW IN A BROWSER

HARNESS IN YOUR PROJECT

What it sees	What you pasted	The repository — it opens the files it decides it needs
What it can run	Nothing	Your shell: <code>uv run pytest</code> , <code>ruff</code> , <code>git</code> , your notebook
Who carries the error	You, by hand, every time	It reads its own traceback and tries again
What persists	One tab, until you close it	<code>AGENTS.md</code> in git — conventions the whole cohort shares
Tools & data	Whatever you retype into the box	MCP servers — <code>goad</code> and <code>codestyle</code> become callable
Your job	Courier, compiler and reviewer	Reviewer — which is the only one that was ever worth your time

The harness is not a detail

- **SWE-agent** (Princeton, 2024) named the *agent-computer interface* and showed that changing only the interface — model held fixed — changes how well the agent performs. [arXiv:2405.15793](https://arxiv.org/abs/2405.15793)
- **mini-swe-agent** is roughly 100 lines of Python with *bash and nothing else* — no tool-calling API at all — and reports over 74% on SWE-bench Verified.

```
# the entire tool surface  
subprocess.run(cmd, shell=True)
```

READ THIS CRITICALLY

That 74% is self-reported by the project. Most head-to-head “harness X beats harness Y on benchmark Z” numbers circulating right now are vendor marketing, two blog-hops from any methodology, and frequently contradict each other. Take the *shape* of the finding — scaffolding matters a lot — and distrust every decimal, including the ones on this slide.

What changes in MADS-DAV

- **goad and codestyle are MCP servers.** From a browser chat they are unreachable — that is the whole reason they exist as servers. In a harness they become tools the agent calls, so the coaching happens where the code is.
- **A pass/fail signal already exists here.** `uv run pytest`, `ruff`, the pre-commit hooks. You have been running them by hand. Hand them to the agent instead.
- **Write the house rules down once.** `uv` only — no `pip`, no `conda`. Work on your own branch, never `main`. Copy a notebook under a new name before editing. That is an `AGENTS.md`, committed, not a paragraph you retype every session.
- **What does not change:** nothing here is graded on the agent's behalf. You defend the flow, the types and the contracts — signatures, `Protocol`s, ABCs — with the agent closed. Not every line; the abstraction.

The landscape, September 2026

COMMERCIAL

Claude Code Anthropic · CLI+IDE

Gemini CLI Google · CLI

Codex CLI OpenAI · CLI, Apache-2.0

Cursor Anysphere · IDE+CLI

Antigravity Google · IDE+CLI

Copilot CLI GitHub · CLI

OPEN SOURCE

opencode MIT · 75+ providers

goose Apache-2.0 · desktop+CLI

pi MIT · minimal, hackable

Kilo Code MIT · VS Code + CLI

Crush Charm · FSL, not OSI

Cline Apache-2.0 · VS Code

Every tool above speaks [MCP](#) — non-negotiable here, because `goad` and `codestyle` are MCP servers. Most read [AGENTS.md](#). **Copilot CLI** needs a paid seat; the free student tier counts, plain Copilot Free does not — see the next slide for what that seat actually gets you.

Read rather than install: [mini-swe-agent](#) is a control, not a daily driver — bash only, ~100 lines, built so a benchmark can hold the scaffold fixed while the model varies. Also: [Aider](#), the original git-native pair programmer, still works but has no native MCP; [hermes-agent](#) (Nous Research, MIT) is a general-purpose agent built on persistent memory and self-written skills — not a coding harness, and not their Hermes *models*.

What I would actually install

I USE THIS DAILY

Claude Code, on a Claude Pro plan — \$20 a month, \$17 if you pay yearly. Claude Code is included in Pro.

It is the one I run every day and the only one I can answer your questions about. If you are going to spend anything on this course, spend it here.

TRY IT — NO PROMISES

Codex CLI — needs a paid ChatGPT plan. Plausibly in the same class — but I have not tested it.

Gemini CLI — free with a personal Google account: 1,000 requests a day on Gemini 3, shell and MCP included. The best free option there is.

Copilot CLI — only if you already hold the GitHub Student seat. Auto model selection only, and agent mode drains the credits fast.

FREE, IF YOU HAVE THE BOX

Ollama behind **opencode** · **pi** · **Crush** · **goose** — a local model, no subscription, nothing leaving your machine. Needs real memory; on a thin laptop it disappoints.

The harness is not the model. Copilot CLI and opencode drive Claude models too — what you pay for is which models the tool may pick and how many turns you get. Prices and student allowances change every few months.

Eight habits worth building now

- **Give it the repo, not a snippet.** Start it inside the project directory and point at real files. Grounded beats plausible.
- **Write `AGENTS.md` once and commit it.** Only what it cannot infer from the code. Under ~200 lines — a bloated file gets ignored.
- **Ask for a plan before code.** Then read the plan and edit it. The plan is the cheapest place to catch a wrong approach.
- **Hand it a pass/fail check.** Without one, “looks done” is the only signal it has, and you are back to being the loop.
- **Branch first. Read the diff, then let it commit.** It writes better messages than you do. Review the diff itself — not the agent's summary of it, written by the same process that wrote the bug.
- **Clear between unrelated tasks.** Long sessions degrade measurably. After two failed corrections, restart with a better prompt.
- **Auto-approve is a fair trade — for the risk you mean.** Git makes a bad edit free to undo. It does nothing about a fetched page telling the agent to read your `.env` and POST it. Sandbox, or keep secrets out of reach.
- **Own the abstraction, not every line.** You need not have typed it. You do need the flow, the signatures, the type hints, and which `Protocol` or ABC a thing satisfies.

Your first thirty minutes

- 01 Install **one** harness. Any of them. Do not comparison-shop first.
- 02 `cd` into your MADS-DAV clone, on your own branch.
- 03 Write `AGENTS.md` : uv only, branch rule, where the data lives, how to run the tests.
- 04 Give it a real task you would otherwise have googled.
- 05 Make it run `uv run pytest` before it is allowed to say it is done.
- 06 Read the diff. Then let it write the commit.
- 07 Clear the context. Next task.

```
$ cd ~/code/MADS-DAV
$ git switch -c my-branch
$ claude          # or codex, opencode, ...

> read AGENTS.md, then fix the
  date parsing in notebooks/02.
  run uv run pytest and keep
  going until it passes.
```

Step 3 is the one people skip and the one that pays.
Everything you write there, you stop retyping forever.

Four things that are genuinely true and inconvenient

- **Context rot is measured, not folklore.** Chroma tested 18 models: performance degrades as input grows, non-uniformly. The 10,000th token is not handled as reliably as the 100th.
- **The lethal trifecta.** Private data + untrusted content + the ability to send something outwards = exfiltration. An agent that fetches one web page has all three. The model cannot reliably detect the attack.
- **Complacency is the documented risk.** Thoughtworks put “complacency with AI-generated code” on *Hold*: bigger diffs, less scrutiny, more duplication, less refactoring.
- **You are a bad judge of your own speedup.** METR's 2025 trial found experienced developers 19% *slower* with AI — while believing afterwards they had been 20% faster. METR now says that number is stale: “it is likely that developers are more sped up from AI tools now.” The 40-point gap between felt and measured is the part no model release fixes.

None of this argues for going back to the browser tab. It argues for a tight loop, small diffs, a real gate, and reading what you ship.

SOURCES ALL LINKS CHECKED SEPTEMBER 2026

Read these, in this order

[Sam Schillace — What is a harness and why do I care?](#)

[Simon Willison — How coding agents work](#)

[Birgitta Böckeler — Harness engineering for coding agent users](#)

[Addy Osmani — Agent harness engineering](#)

[Hugging Face — Agent glossary \(harness vs. scaffold\)](#)

[Anthropic — Claude Code best practices](#)

[agents.md — the cross-tool instruction file](#)

[modelcontextprotocol.io — what MCP actually is](#)

[SWE-agent — agent-computer interfaces \(arXiv:2405.15793\)](#)

[Chroma — Context rot](#)

[Simon Willison — The lethal trifecta](#)

[Simon Willison — Hallucinations in code](#)

[Thoughtworks Radar — Complacency with AI-generated code](#)

[METR — Do AI tools speed up developers? and their 2026 walk-back](#)